

Assignment 2

Lab 3: Control Flow for Data Cleaning

UCI Heart Disease Dataset

Relevant attributes: The dataset contains patient-related clinical variables including age, resting blood pressure (trestbps), cholesterol, maximum heart rate, etc.

```
> # Load dataset
> heart <- read.csv("heart_disease_uci.csv")
> # View structure
> str(heart)
'data.frame':  920 obs. of  16 variables:
 $ id      : int  1 2 3 4 5 6 7 8 9 10 ...
 $ age     : int  63 67 67 37 41 56 62 57 63 53 ...
 $ sex     : chr  "Male" "Male" "Male" "Male" ...
 $ dataset : chr  "Cleveland" "Cleveland" "Cleveland" "Cleveland" ...
 $ cp      : chr  "typical angina" "asymptomatic" "asymptomatic" "non-anginal" ...
 $ trestbps: int  145 160 120 130 130 120 140 120 130 140 ...
 $ chol    : int  233 286 229 250 204 236 268 354 254 203 ...
 $ fbs     : logi  TRUE FALSE FALSE FALSE FALSE FALSE ...
 $ restecg : chr  "lv hypertrophy" "lv hypertrophy" "lv hypertrophy" "normal" ...
 $ thalch  : int  150 108 129 187 172 178 160 163 147 155 ...
 $ exang   : logi  FALSE TRUE TRUE FALSE FALSE FALSE ...
 $ oldpeak : num  2.3 1.5 2.6 3.5 1.4 0.8 3.6 0.6 1.4 3.1 ...
 $ slope   : chr  "downsloping" "flat" "flat" "downsloping" ...
 $ ca      : int  0 3 2 0 0 0 2 0 1 0 ...
 $ thal    : chr  "fixed defect" "normal" "reversable defect" "normal" ...
 $ num     : int  0 2 1 0 0 0 3 0 2 1 ...
```

Task 1 – Create a BP-cleaning function using if-else: Detect negative blood-pressure values and convert them to NA. Detect systolic/resting BP values greater than 250 mmHg and cap them at 250.0 Retain valid BP values without modification.

```
> set.seed(123)
> # Negative BP values
> heart$trestbps[c(5, 20)] <- c(-120, -95)
> # Missing values
> heart$trestbps[c(10, 35)] <- NA
> # Extreme BP values (>300)
> heart$trestbps[c(50, 75)] <- c(320, 450)
> # Zero value for ratio calculation
> heart$trestbps[100] <- 0
> |
```

Task 2 – Implement error handling using tryCatch(): Safely calculate mean BP when missing values are present. Calculate a suitable ratio, such as chol / trestbps. Handle cases where the denominator is zero, NA, or invalid. Generate an informative error/warning message instead of terminating the program.

```
clean_bp <- function(bp)
{
  if(is.na(bp))
  {
    return(NA)
  }
  else if(bp < 0)
  {
    return(NA)
  }
  else if(bp > 250)
  {
    return(250)
  }
  else
  {
    return(bp)
  }
}
# Apply function
heart$BP_Clean <- sapply(heart$trestbps, clean_bp)
```

Task 3 – Compare loop-based and vectorized data cleaning: Use a for loop to identify invalid/outlier BP values. Perform the same operation using vectorized R operations. Measure execution time using system.time() or microbenchmark. Compare the performance of the two approaches.

```
> # Apply function
> heart$BP_Clean <- sapply(heart$trestbps, clean_bp)
> safe_mean <- function(x)
+ {
+   tryCatch(
+     {
+       mean(x, na.rm = TRUE)
+     },
+     warning = function(w)
+     {
+       print(paste("Warning:", w$message))
+       return(NA)
+     },
+     error = function(e)
+     {
+       print(paste("Error:", e$message))
+       return(NA)
+     })
+ }
> mean_bp <- safe_mean(heart$BP_Clean)
> print(mean_bp)
[1] 132.2917
```

```

> safe_ratio <- function(chol, bp)
+ {
+   tryCatch(
+     {
+       if(is.na(bp) || bp == 0 || is.na(chol))
+       {
+         return(NA)
+       }
+       return(chol / bp)
+     },
+     error = function(e)
+     {
+       return(NA)
+     })
+ }
> heart$Chol_BP_Ratio <-
+   mapply(safe_ratio,
+         heart$chol,
+         heart$BP_Clean)
> |

```

Task 4 – Validate the cleaned data: Count missing BP values. Find minimum, maximum, mean, and median BP. Verify that no negative or >250 values remain after cleaning.

```

loop_time <-
  system.time({
    for(i in 1:length(bp_loop))
    {
      if(is.na(bp_loop[i]))
      {
        next
      }

      if(bp_loop[i] < 0)
      {
        bp_loop[i] <- NA
      }
      else if(bp_loop[i] > 250)
      {
        bp_loop[i] <- 250
      }
    }
  })
bp_vector <- heart$trestbps
vector_time <-
  system.time({
    bp_vector[bp_vector < 0] <- NA
    bp_vector[bp_vector > 250] <- 250
  })
print(loop_time)
  user  system elapsed
0.00    0.00    0.01

```

```

> print(vector_time)
  user  system elapsed
    0      0      0
> cat("\nMissing Values :",
+     sum(is.na(bp_vector)))

Missing Values : 63
> cat("\nMinimum BP :",
+     min(bp_vector, na.rm = TRUE))

Minimum BP : 0
> cat("\nMaximum BP :",
+     max(bp_vector, na.rm = TRUE))

Maximum BP : 250
> cat("\nMean BP :",
+     mean(bp_vector, na.rm = TRUE))

Mean BP : 132.2917
> cat("\nMedian BP :",
+     median(bp_vector, na.rm = TRUE))

Median BP : 130
> cat("\nNegative Values Remaining :",
+     sum(bp_vector < 0, na.rm = TRUE))

Negative Values Remaining : 0
> cat("\nValues >250 Remaining :",
+     sum(bp_vector > 250, na.rm = TRUE))

Values >250 Remaining : 0

```

```

> heart$trestbps <- bp_vector
> write.csv(
+   heart,
+   "cleaned_heart_data.csv",
+   row.names = FALSE
+ )
> |

```

The vectorized approach was more efficient than the loop-based approach, as it executed in less time by operating on the entire vector simultaneously rather than processing each element individually. Vectorized operations are optimized internally in R, making them faster, more concise, and better suited for large datasets. Therefore, vectorized programming is the preferred approach for data cleaning tasks whenever possible.